CAVE

The Complete System Guide

Compressed Atomic Verb Expressions

Repository version 0.28.1

About this book

This book consolidates the implemented CAVE system and its normative repository specification into one continuous technical guide. The specification skills remain authoritative for exact normative wording and section numbers; this book explains how the pieces work together.

Build target: Typst 0.15.0. Project version: 0.28.1.

Contents

1. What CAVE Is

CAVE stands for Compressed Atomic Verb Expressions. It is a small line-oriented language and a local system for recording knowledge as atomic claims. A claim is easy for a person or language model to write, easy to diff in source control, formal enough to query, and persistent enough to retain changes in belief over time.

The smallest useful statement has three parts: a subject, an uppercase verb, and an object. CAVE extends that shape with attributes, values, confidence, contexts, tags, comments, uncertainty, temporal scope, and lineage. The language remains deliberately small; system behavior is built by composing claims rather than by adding unrelated syntactic forms.

```
subject VERB object
subject HAS attribute: value

service/auth USES jwt
auth/middleware HAS bug: token-expiry #security
server IS NOT compromised @ 90%
```

CAVE is not only a notation. The repository implements a complete loop around the notation: parsing, canonicalization, SQLite persistence, graph querying, deterministic and language-model ingestion, rules, governed actions, contradiction resolution, alias discovery, synchronization, automations, an MCP server, a read-only web view, evaluation, and cited reports.

Design boundary
CAVE targets a single-machine, inspectable knowledge system. It favors plain text, SQLite, deterministic transforms, and explicit provenance over a distributed platform or a hidden vector-only memory layer.

2. Design Principles

The system is guided by six constraints. Each constraint removes a common source of accidental complexity.

Principle	Consequence
Caveman-simple	Short direct statements; low ceremony.
Atomic	One independently useful claim per line.
Composable	Claims can qualify, support, contradict, or supersede other claims.
Queryable	Every persisted row has stable structural fields.
Persistent	Belief changes append; history is not overwritten.
LLM-friendly	Extraction can preserve uncertainty, estimates, and source context.

The strongest rule is that new core syntax is added only when semantics require it. Extension verbs, shape declarations, source policy, actions, automations, and rules are represented as ordinary claims wherever possible. This keeps the grammar compact and moves policy into queryable data.

CAVE also separates three concerns that are frequently conflated: transaction time says when a store learned something; valid time says when a claim applies in the modeled world; confidence says how strongly the claim is believed. Keeping these axes distinct enables historical and temporal questions without snapshots or destructive edits.

3. The Claim Model

Every CAVE line denotes an immutable claim. Conceptually, a claim contains a subject, verb, payload, polarity, and metadata. The payload is either a relation object or an attribute/value pair.

```
claim = <subject, verb, payload, negated, metadata>
metadata = <confidence, contexts, tags, uncertainty, importance,
            comment, transaction identity, source provenance>
```

Relational claims connect two terms. Attribute claims attach a named property to a subject. Metric claims use IS with a numeric or temporal value. The same storage and query machinery handles all three.

```
repo CONTAINS packages/core
pool HAS max: 20 conn
revenue IS ~20B USD/yr +/- 2B USD/yr @2026-Q1 @ 90%
```

A claim is immutable after append. The current state of a fact is resolved from its belief series. Contradictory series can coexist because CAVE treats disagreement as data rather than as a write-time error.

The implementation recognizes progressively richer usage layers: CAVE-Lite for bare triples, CAVE-Core for triples plus metadata, and CAVE-Full for qualified claims, continuation, uncertainty, history, and inverse-aware reads.

4. Core Syntax

Canonical CAVE has two primary line shapes. The colon in an attribute claim is significant: it separates the attribute name from its value and prevents ambiguity with a relation object.

```
subject VERB [NOT] object [qualifiers] [; comment]
subject HAS attribute: value [+/- delta [(N sigma)]] [qualifiers] [; comment]
```

Token	Meaning
@context	Scope, source, location, environment, or valid-time context.
@ 90%	Epistemic confidence; the space after @ distinguishes it from context.
#tag	Flat tag.
#key:value	Scoped tag.
+/-	Numeric uncertainty.
≈	Approximate value marker.
!	Importance marker.
;	Persistent human comment.
NOT	Explicit logical negation.
->	Trajectory between numeric endpoints.

Entities are compact names. Slash introduces scope, kebab-case is preferred within a segment, and proper nouns retain their casing. Exact code-like text belongs in backticks; natural-language literals belong in double quotes.

```
auth/middleware
auth/middleware/token-check
PostgreSQL
`ECONNRESET`
"install dependencies"
```

Pronouns are intentionally poor entity names. Extraction should resolve “it” or “the component” to a stable term. Reusing the same entity spelling is more valuable than inventing stylistic variants.

5. Verbs, Extensions, and Inverses

IS and HAS are sufficient to bootstrap the language. Standard verbs improve graph quality for common relations such as causation, dependency, structure, ordering, comparison, and provenance.

Family	Typical verbs
Identity and taxonomy	IS, EXTENDS, ALIAS, LIKE, EXISTS
Causation and change	CAUSE, FIX, BECOMES
Dependency and production	NEEDS, USES, YIELDS, ENABLES, BLOCKS
Structure and ordering	CONTAINS, PRECEDES, EXCEEDS
Evidence and qualification	BECAUSE, VIA, WHEN, UNLESS

Domain verbs are declared in-band. A declaration is itself a claim, which means vocabulary changes are attributable, reviewable, and append-only.

```
WORKS-AT IS verb ; X is employed by organization Y
WORKS-AT REVERSE EMPLOYS
```

REVERSE declares two readable names for one fact. Only the primary direction is stored. A write or query using the inverse is canonicalized by swapping endpoints and substituting the primary verb. Forward and inverse readings share one claim key, one history, and one physical row.

```
team/acme EMPLOYS alice      ; input form
alice WORKS-AT team/acme      ; canonical stored form
```

Why inverses are views
Materializing both directions would double rows, split belief histories, and duplicate contradiction work. Query-time inverse views preserve one fact with two human-readable directions.

Verb names can evolve without rewriting those rows. A directional lifecycle declaration deprecates the old authoring name while keeping it as the stable storage identity:

```
WORKS-AT RENAMED-TO EMPLOYED-BY
alice EMPLOYED-BY team/acme
```

Both spellings write and query the stored verb WORKS-AT, so claims before and after the rename share one claim key and history. The replacement must be declared before first use. Linear rename chains are allowed; branches, joins, cycles, and collisions with an existing verb identity are rejected. As-of queries reconstruct lifecycle declarations at the same transaction boundary, and renaming either side of a REVERSE pair preserves its direction.

6. Indentation and Claim Graphs

Indentation expresses relationships between claims without introducing a second document format. CAVE distinguishes qualifier children from continuation lines.

```
server CAUSE crash @ 80%
  WHEN load EXCEEDS 1000 req/s
  WHEN NOT cache/enabled
```

Qualifier children create edges from the parent claim to child claims. Typical edge roles are WHEN, UNLESS, BECAUSE, VIA, and QUALIFIES. They encode conditions and lineage while preserving each child as an independently stored claim.

```
monorepo CONTAINS packages/api
  CONTAINS packages/web
  CONTAINS packages/core
  PART-OF org/monorepos
```

Continuation lines inherit an endpoint from the parent and expand to sibling claims. An inverse continuation can place the inherited term in object position. The canonicalization phase resolves this before storage.

The resulting store is a graph of immutable claim rows plus explicit edge rows. Text is a readable tree projection of that graph; exports may restate a shared row when several parents cite it.

7. Values, Units, Confidence, and Uncertainty

CAVE separates uncertainty in a measured value from uncertainty in belief. A numeric delta describes dispersion or tolerance in the value. Confidence describes the strength of evidence for the claim.

```
OpenAI HAS revenue: ~20B USD/yr +/- 2B USD/yr @2026-Q1 @ 90%
model HAS latency: 210ms +/- 15ms (1 sigma) @lab
```

The store keeps both raw value text and parsed numeric fields when possible. Multipliers such as K, M, and B normalize to ordinary numbers. Units remain explicit and comparisons require compatible units.

The probabilistic layer can fuse independent numeric estimates using precision-weighted averaging, with confidence acting as a weight. Conditional confidence can use noisy-AND: a claim at 80 percent conditioned on evidence at 60 percent yields 48 percent effective confidence when independence is an explicit assumption.

Competing hypotheses should normally be represented by different cause entities, not by repeatedly overwriting one opaque claim. Evidence then updates each hypothesis through new appends.

```
memory-leak CAUSE app/crash @ 50%
deadlock CAUSE app/crash @ 30%
oom-killer CAUSE app/crash @ 20%
```

8. Time: Transaction and Valid Time

CAVE has two orthogonal time axes. Transaction time is generated by the store for every append and reconstructs what the store believed at a past boundary. Valid time is authored in the claim and describes when the fact applies in the modeled world.

```
jan WORKS-AT textile-mill @1950..1974
jan WORKS-AT railways @1975..
mill/wage IS 200 -> 900 PLN/mo @1950..1974
```

A time-range context uses two dots. Ranges may be closed or open-ended. A trajectory value uses two numeric endpoints and is interpolated linearly at query time. Timeless claims continue to match any valid-time query.

The CLI exposes transaction time with –as-of and valid time with –at. Because the filters compose, CAVE can answer questions such as: what did the store believe last year about the wage in 1962? No snapshots are required because all prior rows remain present.

```
cave query --db family.db "jan WORKS-AT ?where" --at 1960
cave query --db family.db "mill/wage IS" --at 1962
cave query --db family.db "mill/wage IS" --at 1962 --as-of 2025-12-31
```

9. Append-Only Belief Evolution

The store never mutates a claim row. A new assessment appends a newer row in the same belief series. Current belief within a series is the row with the greatest transaction identifier.

```
Anthropic HAS ipo-timing: 2026-H2 @ 40%
Anthropic HAS ipo-timing: 2026-H2 @ 65% ; after CFO statement
Anthropic HAS ipo-timing: 2026-H2 @ 35% ; market worsened
```

The claim key defines which rows belong to one evolving fact. For a relation it includes canonical subject, verb, object, polarity, and non-source contexts. For an attribute claim, the value is outside the key so the property can change without becoming a different question.

Retraction is an append at zero confidence. Retraction removes current support but preserves the earlier assertion. Explicit negation is stronger and records the opposite proposition.

```
server IS compromised @ 0% ; retracted
server IS NOT compromised @ 90% ; clean forensic scan
```

Contradictions are legal at write time. CAVE does not impose global logical consistency because disagreements between sources, models, and human reviewers are often the knowledge that matters.

10. Provenance and Source Stamping

Every append surface should identify who or what wrote a claim. Source provenance is represented by src-prefixed contexts, so it participates in claim identity and is queryable like any other context.

Surface	Typical stamp
Interactive CLI	@src:cli
MCP append	@src:agent/
LLM ingest	@src:ingest
Structured connector	@src:connect//
Rule conclusion	@src:rule/
Action effect	@src:action/
Automation agent output	@src:automation/

An explicitly authored source normally wins over automatic stamping. Lifecycle stamps used for connect records, rule conclusions, and action effects are added even when another source is present, because the system must find those rows later for attribution or retraction.

Stamping occurs before the claim key is computed. This intentionally gives different actors separate belief series. Restating a claim with another actor's source context is the explicit way to supersede or retract that actor's series.

11. SQLite Storage Model

The physical design is a compact relational schema. Claims occupy the central table; contexts, tags, and claim-to-claim edges live in side tables. Full-text search indexes human-readable fields.

```
cave_claim(id, tx, subject, verb, negated,
           object, attribute, value_text, value_num, value_unit,
           value_approx, delta_text, delta_num, delta_unit,
           sigma_level, conf, importance, comment, raw_line, claim_key)

cave_context(claim_id, context)
cave_tag(claim_id, key, value)
cave_edge(parent_id, role, child_id)
cave_fts(claim_id, subject, verb, object, attribute, value_text, comment, raw_line)
```

The canonicalization pipeline normalizes verb casing, resolves inverse verbs, expands continuation lines, normalizes entity whitespace, parses confidence and numeric multipliers, splits tags, computes the canonical claim key, and persists side-table rows. raw_line remains available for fidelity and diagnostics.

The core current-belief query joins each claim key to its maximum transaction. As-of queries apply the same grouping after restricting rows to a transaction boundary. Inverse reads use subject and object indexes without adding rows.

Portability
Canonical CAVE text is the backup and interchange form. SQLite is the efficient working representation, not a proprietary source of truth.

12. Permanent History and Sensitive Data

Retraction changes current belief by appending a zero-confidence row. It does not erase the earlier row, authored text, metadata, search index, full export, sync peer, or backup. A current-only export is a compact view, not a sanitizer: the surviving rows may themselves carry sensitive subjects, values, contexts, or comments.

CAVE deliberately provides no claim-level redact or forget command. A local rewrite could not guarantee erasure across SQLite remnants, FTS tables, transaction files, copied databases, annotated exports, version-control history, peers, backups, snapshots, and storage media; an older peer could also reintroduce the globally identified row. Keep credentials and selectively erasable personal data outside the store.

Accidental secret response
Rotate or revoke the secret first. Stop writers and sync, inventory every database/export/clone/backup/snapshot, rebuild from reviewed safe input, verify the replacement independently, then explicitly destroy or expire every affected copy using the storage provider's confirmed procedure. Never merge an affected store into the replacement.

13. CAVE-Q Query Language

CAVE-Q reuses the claim shape as a graph pattern language. Slots beginning with ? are named variables. An underscore is an anonymous wildcard. A plus suffix requests one-or-more transitive hops.

```
?x USES jwt
?x HAS bug: ?bug #security
?cause CAUSE app/crash
terrier EXTENDS+ animal
_ USES jwt
```

Filters restrict matched rows by confidence, tag, context, value, unit, transaction, or other stored fields. Inverse verbs compile to the same canonical rows as their primary direction. Deprecated and preferred verb spellings declared with RENAMED-TO compile to one stable storage verb and belief history.

```
WHERE conf >= 0.8
WHERE tag = security
WHERE context = production
WHERE value > 1000 req/s
WHERE tx <= 2026-01-15
```

Query modes distinguish current resolved rows from full history. –as-of reconstructs the store's belief state at a date, timestamp, or UUIDv7 transaction. –at anchors valid time. –aliases widens entity positions through the current alias closure. –resolve selects a trusted winner among competing source series.

SQL remains available for advanced analysis. CAVE-Q is the ergonomic graph layer; SQL is the transparent escape hatch.

14. Aliases and Entity Resolution

ALIAS asserts that two names denote the same entity. Alias-aware querying computes an opt-in undirected transitive closure over current positive entity-to-entity ALIAS claims.

```
postgres ALIAS postgresql
billing USES postgres
analytics USES postgresql
```

With aliases enabled, a query for systems using postgres can match both spellings. The store does not rewrite rows or pick a canonical name. It widens matching and returns the stored spelling, preserving separate histories and making disagreements visible.

Unmerge is a retraction of the alias claim. Negated, retracted, or literal-ended ALIAS claims do not connect the closure. Values, attributes, and verbs are not subject to alias expansion.

cave suggest-alias discovers candidate pairs using explainable string and graph signals such as separator drift, segment containment, edit distance, shared rare attributes, and relation overlap. Suggestions are low-confidence questions. Human confirmation or rejection is appended as ordinary ALIAS knowledge, so the decision persists.

15. Shape Expectations and Knowledge Health

CAVE records expected structure as claims rather than in an external schema language. EXPECTS declares that instances of a type should carry an attribute or participate in a relation.

```
EXPECTS IS verb
service EXPECTS owner
service EXPECTS repo
service EXPECTS USES
team EXPECTS PART-OF
team EXPECTS PART-OF #cardinality:one
service EXPECTS latency #unit:ms
```

Expectations apply through the IS and EXTENDS taxonomy. A direct instance of a subtype inherits expectations from ancestor types. Attribute expectations look for a positive HAS attribute claim. Relation expectations account for inverse direction. The compatible default is one or more matches. `#cardinality:one` requires exactly one current match, chiefly for relation endpoints because attribute claim keys already select one current value. `#unit:ms` requires an attribute value whose normalized unit is exactly `ms`; unitless values and implicit conversions do not satisfy it.

cave check is a read-only health report. It lists unsatisfied expectations with observed counts and units, stale beliefs, medium-confidence review candidates, alias disagreements, and aggregate coverage. Violations make the command fail; advisory sections identify the frontier without blocking normal reads.

Write gating reuses the same checks transactionally. cave add –check and actions can append, evaluate newly introduced violations, and roll back only if the operation makes health worse. Existing violations do not prevent unrelated progress.

16. Ingestion from Prose

cave ingest turns files or web pages into claims using any headless agent that can either call CAVE MCP tools or return CAVE text on stdout. Domain instructions steer modeling choices while the engine owns persistence and provenance.

- One claim per line; split compound statements.
- Resolve pronouns and unstable descriptions to concrete entities.
- Prefer chosen decisions over the full debate, while retaining important rejected alternatives.
- Keep code identifiers and exact errors in backticks.
- Drop conversational wrapper and repeated explanation.
- Preserve uncertainty and source context.
- Choose granularity that supports a future query or action.

Input files are digested so unchanged content is skipped on later runs. Batching limits prompt size. A source can be a path, glob, or URL; web content is fetched and readability-extracted before the agent sees it.

```
cave ingest --db lore.db "docs/**/*.md" https://example.com/design \
  --instructions domain.md \
  --agent 'claude -p --mcp-config {mcp-config} --allowedTools "mcp__cave__*"'
```

Shell template substitutions are quoted by the implementation. Placeholders such as {db}, {prompt-file}, and {mcp-config} should be written bare so paths with spaces or shell metacharacters remain one argument.

17. Deterministic Structured Ingestion

Structured records should not consume language-model tokens. cave connect maps CSV, TSV, JSON, JSONL, SQLite queries, or structured URLs through a CAVE template containing ?field variables.

```
; people.map.cave
WORKS-AT IS verb
WORKS-AT REVERSE EMPLOYS

?id IS person
?id HAS name: ?name
?id HAS age: ?age
?id WORKS-AT ?company
```

Blocks without variables form a prelude and append once per run. Blocks with variables instantiate per record. Missing or empty fields drop the affected line and its children. Substituted values are formatted deterministically as numbers, safe atoms, parseable values, or quoted literals.

Each record receives a stable connect identity and digest. Unchanged records skip. Changed records append new claims and retract prior record-owned claims no longer produced. –prune retracts records removed from the source. The lifecycle source stamp makes this ownership explicit.

–watch reruns when a file changes. –query temporarily overlays mapped claims inside a rolled-back transaction, allowing a CAVE-Q query across local and external data without persisting the external rows.

18. Evaluation and Quality Control

Extraction quality must be measurable. cave eval runs fixture suites containing a source, expected golden claims, and optional CAVE-Q answers. It can repeat each case against any agent in fresh temporary stores.

Strict scoring compares claim keys, tolerates configured numeric value differences, and ignores actor stamps when appropriate. The report includes precision, recall, F1, misses, extras, and failed query bindings. An optional judge agent may pair semantically equivalent leftovers without replacing the strict score.

```
cave eval examples/eval \
  --agent 'claude -p --mcp-config {mcp-config}' \
  --runs 3 --min 90%
```

A minimum score turns the suite into a CI gate. This makes prompt, model, and instruction changes falsifiable instead of anecdotal. Reconstruction policies can be evaluated with the same framework by scoring the claims collected from a loop.

19. Rules and Derivation

Rules conclude claims that nobody wrote directly. A rule is a comma-separated conjunction of CAVE-Q premises followed by one conclusion. The only rule-specific token is =>.

```
?a PARENT-OF ?b, ?b PARENT-OF ?c => ?a GRANDPARENT-OF ?c
?x HAS age: ?a, ?a < 18 => ?x NEEDS guardian
```

Premises evaluate left to right. They may be patterns or constraints. Variables in a constraint must already be bound. Every conclusion variable must be bound by a premise. Explicit NOT matches negated claims; CAVE does not use negation-as-failure.

Rules are stored in-band as claims under rule/. Derived rows receive a rule lifecycle source stamp, BECAUSE edges to exact premise rows, and a VIA edge to the rule declaration. Confidence composes from premise confidence and an optional rule factor using noisy-AND.

cave derive is incremental and idempotent. Watermarks avoid reevaluating unaffected rules. Re-running appends only changed conclusions. If support disappears, well-founded support tracking retracts conclusions that no longer have a valid derivation.

20. Governed Actions

Rules fire autonomously. Actions let a caller execute a named write path with parameters and validated preconditions. The action body reuses rule syntax but treats bare variables on the left as caller-supplied parameters and permits multiple effects on the right.

```
action/mark-deployed HAS action: `?service, ?version,
  ?service IS service =>
  ?service HAS deployed-version: ?version`
```

Execution resolves the current action declaration, validates parameters, evaluates preconditions against current positive beliefs, requires deterministic bindings for effect variables, canonicalizes all effects, and appends them atomically. Effects are stamped @src:action/, linked by BECAUSE and VIA edges, and deduplicated when already current.

Actions run inside the shape gate by default. A failed precondition or introduced violation produces no append. This is the preferred write vocabulary for agents because it replaces unconstrained freeform changes with named operations and explicit schemas.

An action may name an out-of-band hook. Hook commands remain in configuration, never in the store. They run after a successful commit, receive canonical claims on stdin, and cannot roll back recorded knowledge if the external side effect fails.

21. Contradiction Resolution

Default reads preserve all coexisting claims. Resolution is an explicit read mode that chooses one stored winner for each contested fact without rewriting history.

Rows enter the same resolution group when they describe the same subject, verb, payload identity, and non-source contexts after removing source contexts and polarity. Source differences and positive versus negative answers therefore compete; production and staging facts do not.

Priority	Rule
1. Precedence class	Higher source tier wins.
2. Effective confidence	Stored confidence multiplied by source reliability.
3. Transaction	Newest row breaks remaining ties.

Source policy is ordinary knowledge. source/ HAS precedence and source/ HAS reliability claims use path-prefix matching, with the most specific current declaration applying. A human CLI correction can therefore outrank a newer machine ingest row, while recency still decides within one tier.

```
source/cli HAS precedence: 4
source/agent HAS precedence: 3
source HAS precedence: 2
source/rule HAS precedence: 1
source/ingest HAS reliability: 80%
```

cave resolve explains the ranking and cave query –resolve returns only winners. The original candidates remain available through normal queries and full history.

22. Reconstruction and Agent Memory

A direct query requires the caller to know what pattern to ask. cave reconstruct starts from an entity or symptom and walks the graph best-first, collecting related claims until a budget or stopping condition is reached.

The deterministic policy scores frontier entities using relation direction, confidence, recency, and novelty. –trace exposes every expansion and score. An optional agent policy receives the collected claims and frontier at each step and chooses the next cue or STOP.

```
cave reconstruct --db incident.db checkout/errors --trace
cave reconstruct --db incident.db checkout/errors \
  --query "what caused the failures?" --agent 'claude -p'
```

The heuristic is a baseline rather than a hidden fallback. Reconstruction fixtures allow the language-model policy and deterministic policy to be scored by the same claim-key F1. This makes claims about better memory retrieval testable.

Reconstruction differs from vector retrieval: it follows explicit typed relations and inverse views, returns source claims, and exposes the path that brought each claim into context.

23. Store Synchronization and Git

CAVE stores merge by immutable row identity. UUIDv7 claim ids and edge identities make sync a set union: present rows skip, absent rows insert, and contradictory knowledge does not create a merge conflict because contradictions are legal and resolved at read time.

```
cave sync --db main.db laptop.db
cave export --db laptop.db --tx | cave sync --db main.db - --as laptop
```

A successful merge can append a SYNCED-INTO claim naming origin and target labels. The receive rule ensures later local appends sort after merged history even when clocks differ.

Transaction annotations are full-line comments of the form ;@ immediately above a claim. cave export –tx emits them and cave sync replays them with original identity. Plain import ignores the annotations and performs an ordinary replay.

The recommended Git workflow commits the annotated text export, not the SQLite file. A branch rebuilds a private store by syncing the export with –no-record, performs normal appends, re-exports, and reviews the textual diff. Text conflicts are resolved by syncing both exports into a fresh store and exporting the union, which can be configured as a merge driver.

24. Automations and the Closed Loop

Automations watch belief changes and connect triggers to steps. A declaration contains CAVE-Q trigger premises on the left and action names, hook names, or agent prompt templates on the right.

```
automation/page-on-spike HAS automation: `
  ?svc IS service,
  ?svc HAS error-rate: ?r,
  ?r > 0.05 =>
  action/open-incident,
  hook/page,
  "investigate the spike on ?svc"`
```

A solution fires only when at least one supporting premise row is newer than the automation watermark. Bookkeeping rows and an automation's own outputs are excluded so the loop cannot wake itself forever. Other automations' output remains eligible, enabling chains.

The settle cycle records a watermark before executing steps, fires incremental rules, executes governed actions, runs configured hooks, and records an agent's CAVE reply with automation provenance. Idempotent write paths ensure the cycle converges.

With cave connect –watch feeding source changes and cave automate watching store changes, the single-machine loop is: sense, model, conclude, act, record, and inspect.

25. Human and Machine Interfaces

The CLI is the broad operator surface. MCP exposes the store to agents. cave serve and cave report provide human-facing read surfaces.

Surface	Purpose
cave mcp	Query, append, fuse, reconstruct, and generated action tools for MCP clients.
cave serve	Local read-only self-contained HTML dashboard.
cave report	Render Markdown templates with CAVE-Q values and claim citations.
cave highlight	Tree-sitter-driven terminal syntax highlighting.
VS Code extension	Semantic tokens from the same tree-sitter query.

cave serve provides knowledge-health tiles, entity 360 pages, forward and inverse relations, topic browsing, alias closure, full-text search, belief-history timelines, and BECAUSE/VIA lineage trees. It serves localhost by default and permits GET only.

cave report evaluates fenced cave-q blocks and inline query splices in a Markdown template. Rendered facts carry footnotes with canonical claim text, date, and claim key. Ambiguous inline values fail unless resolution is explicitly requested, preventing a report from silently selecting a contested fact.

The MCP server can be read-only or tool-scoped. It dynamically generates `act_<name>` tools from current action declarations, giving an agent a governed, inspectable write vocabulary.

26. Package Architecture

The repository is a pnpm TypeScript monorepo. Packages form a dependency ladder from domain types to user surfaces. The separation keeps parsing, storage, policy, and orchestration independently testable.

```
@cavelang/core
  -> parser
  -> canonical
  -> store
  -> query
  -> shape
  -> scenario
  -> solver
  -> solver-z3
  -> connect
  -> fusion
  -> rules
  -> act
  -> sync
  -> loop
  -> automate
  -> view
  -> mcp
  -> ingest
  -> eval
  -> tree-sitter-cave
  -> highlight
  -> cli
```

Layer	Responsibilities
Core	Claim/value types, units, UUIDv7 transactions, claim keys.
Parser	Line grammar, metadata, indentation, rule/action bodies.
Canonical	Inverse registry, continuation expansion, stable emission.
Store	SQLite schema, append/import/export, provenance, edges.
Query	CAVE-Q, filters, transitive paths, aliases, temporal reads.
Formal reasoning	Typed scenario snapshots, solver-neutral exact models, bounded verification workflows, optional Z3 search.
Policy	Shape checks, fusion, resolution, derivation, actions.
Orchestration	Connect, ingest, reconstruction, sync, automations.
Presentation	MCP, CLI, reports, web view, highlighting.

A tree-sitter grammar is shared across terminal highlighting, the website, the editor extension, and tree-sitter-native editors. This avoids syntax drift between the parser and presentation surfaces.

27. Formal Reasoning and Scenario Inputs

CAVE-Q retrieves beliefs and rules derive claims, but neither searches a space of possible assignments. The optional formal-reasoning layer handles feasibility, optimization, and proved infeasibility without turning a hypothetical model into durable knowledge.

`@cavelang/scenario` binds explicit CAVE-Q inputs against a frozen transaction-time and valid-time snapshot. It applies hypothetical CAVE claims inside a savepoint, converts authored numeric values and units exactly, records the supporting belief row identities, and rolls the overlay back before any evaluator runs. Missing, contested, retracted, and multiple values require declared policies; no integration silently chooses the first match.

`@cavelang/solver` is a dependency-free, solver-neutral TypeScript model for Boolean, bounded integer, exact real, and finite-enum variables. It distinguishes hard constraints, explicitly weighted soft constraints, and lexicographically ordered objectives. Its workflow API gives feasibility, optimization, counterexample, and bounded sensitivity distinct semantics while sharing one validated model, snapshot context, resource limits, and result vocabulary. Results are disjoint: satisfied and optimal carry assignments, unsatisfied requires a proof of infeasibility, and timeout or backend failure remains unknown. CAVE confidence never becomes an optimization weight implicitly.

Workflow assignments are deterministic: variables are ordered by stable ID, with false before true, smaller exact numbers first, and enum values in lexical order. Authored optimization objectives remain first, explicit soft weights follow, and generated tie-break objectives come last. Sensitivity checks only an explicit typed sample list; its report identifies adjacent assignment transitions and contiguous unknown regions rather than interpolating through timeouts.

`@cavelang/solver-z3` is the optional Node.js adapter to the official threaded Z3 WebAssembly package. It loads lazily, queues checks through one process runtime, tracks named hard constraints for unsatisfiable cores, preserves exact rational arithmetic, applies bounded execution, and requires explicit worker shutdown. Its separate `cave-solver-workflow architecture` binary is an allowlisted fixture that accepts bounded typed flags but no raw model or SMT-LIB input. The normal CAVE CLI, MCP server, browser playground, and knowledge kernel do not depend on Z3.

Solver explanations map assignments, evaluated constraints, objective contributions, and non-minimal unsatisfiable cores back to stable model locations, scenario inputs, and exact CAVE evidence rows. Counterexample reports also state the assumptions, bounded domains, and theories in scope. The model digest, solver version, resource limits, and frozen snapshot travel with the report. Rendering an explanation is read-only. An explicit atomic and idempotent record transition can preserve the immutable run, but recommendations, human decisions, action audit records, and external-effect audit records use separate versioned identities. Replay reports model or solver incompatibility instead of silently re-evaluating. `actProposal` still rechecks the current action declaration, parameters, premises, shape gate, and transaction boundary before a proposed decision can write.

> **Boundary**
> A solver proves statements only inside the selected model and snapshot. Optimal means optimal under those declared inputs and objectives, not objectively best in the world.

28. End-to-End Example: Architecture Decision

CAVE can model a decision such as monolith versus microservices by separating inputs, evidence, derived effects, and the decision itself. The system does not need a special decision object; ordinary claims plus rules and an action are enough.

```
system/shop HAS team-size: 6 people
system/shop HAS deployment-frequency: 3 /wk
system/shop HAS domain-count: 4
system/shop HAS operational-maturity: low

architecture/monolith ENABLES simple-deploy
architecture/microservices NEEDS platform-team
platform-team NEEDS team-size: 8 people

?sys HAS team-size: ?n, ?n < 8 => ?sys NEEDS low-ops-overhead
?sys NEEDS low-ops-overhead => architecture/monolith FITS ?sys @ 80%
```

A report can query the current inputs, show the rules and evidence that support each option, and cite the exact claims. An action such as action/select-architecture can require that evaluation inputs exist and append the chosen architecture plus rationale lineage.

As circumstances change, new input claims append. Derivation recalculates recommendations, but the prior decision remains historically visible. Resolution can prefer an explicit human selection over a generated recommendation. Valid-time contexts can describe planned future team size without overwriting current staffing. If choices interact through hard capacity or isolation constraints, the same inputs can bind through `@cavelang/scenario` into an exact portable solver model; simple weighted guidance should remain ordinary deterministic evaluation.

Limiting factor
CAVE can make the reasoning explicit and reproducible, but it cannot create reliable domain weights from nothing. The quality of a decision model is bounded by the declared factors, evidence, and rules. The next concrete step is to encode one narrow decision with measurable inputs and test it against known cases.

29. Operational Guidance

- Keep canonical text under version control; treat SQLite as a working index.
- Declare domain verbs and inverses in a small prelude.
- Use source contexts consistently so resolution policy can distinguish actors.
- Prefer actions over freeform agent appends for consequential writes.
- Run cave check in CI and use cave eval for extraction regressions.
- Use –resolve only where a single winner is required; keep default reads plural.
- Use –as-of and –at explicitly in reports that depend on time.
- Keep hooks and agent commands out of the store; claims may name them but never contain executable configuration.
- Back up with canonical export and periodically test restore or sync into a fresh database.

Security boundaries are intentionally visible. MCP tool scoping separates reads, ephemeral evaluation, durable recording, and effect-capable actions; exact tool allowlists can narrow further. Actions validate parameters and preconditions. Hook substitution is shell-quoted, but hook configuration remains executable operator-controlled code and should be reviewed accordingly. The read-only server should remain bound to localhost unless placed behind an appropriate access layer.

Performance is designed for local SQLite scale. Indexes support direct entity, verb, object, attribute, confidence, context, tag, and full-text access. Transitive graph queries and large alias closures can become the limiting factor before ordinary claim reads. Measure on representative stores before adding distributed infrastructure.

30. Draft and Deliberately Unfinished Areas

The specification marks sections as normative, legacy, draft, or non-normative. Implemented rule syntax and temporal trajectories have graduated from the draft unified grammar. General reification, variables in ordinary stored claim lines, and arbitrary temporal functions remain draft.

```
[server CAUSE crash] WHEN load EXCEEDS 1000 req/s   ; draft reification
revenue IS (t -> 20B * 1.25^(t - 2025)) USD/yr ; draft function
```

The unified-grammar idea is that facts, queries, and rules share one triple structure and differ primarily by binding state: all slots bound is a fact, some variables is a query, and variables plus => form a rule. Features graduate only after parser and evaluator implementations prove the semantics.

Open design items and suspected bugs are tracked in TODO.md and BUGS.md. The project treats incompleteness as explicit data rather than filling gaps with unimplemented promises.

31. CLI Field Guide

Command	Primary use
cave parse	Validate and summarize a CAVE document.
cave add / import	Append authored or replayed claims.
cave export	Emit canonical text, current view, or tx-annotated replica.
cave query	Run CAVE-Q with filters, aliases, resolve, as-of, and valid time.
cave check	Knowledge health and shape coverage.
cave ingest	LLM extraction from prose and web pages.
cave connect	Deterministic structured mapping and watch mode.
cave eval	Extraction and reconstruction evaluation.
cave derive	Incremental rule materialization.
cave act	Execute a governed named write.
cave resolve	Explain contradiction winners.
cave suggest-alias	Find potential duplicate entity names.
cave reconstruct	Best-first related-claim reconstruction.
cave sync	Merge stores or annotated text by row identity.
cave automate	Watch belief events and settle rules/actions/hooks/agents.
cave serve	Read-only local knowledge browser.
cave report	Cited Markdown report rendering.
cave mcp	Serve tools to MCP clients.
cave highlight	Syntax-highlight CAVE text.
cave-solver-workflow	Run the optional allowlisted Z3 architecture fixture.

Run pnpm exec cave help for the exact versioned option set. Package READMEs contain surface-specific examples, while the four specification skills preserve normative section numbering.

32. Compact Reference

```
subject VERB [NOT] object [@context...] [#tag[:value]...] [@ N%] [!] [; comment]
subject HAS attribute: value [+/- delta [(N sigma)]] [@context...] [#tag...] [@ N%]

VERB REVERSE INVERSE-VERB
OLD-VERB RENAMED-TO NEW-VERB

parent VERB object
  WHEN condition
  BECAUSE evidence
  CONTAINS sibling-object

?x VERB ?y
?x VERB+ ?y
pattern, pattern, constraint => conclusion

@production       context
@ 80%             confidence
@2025..2028       valid-time range
20 -> 40 USD/yr   trajectory
@ 0%              retraction
VERB NOT           explicit negation
#topic:auth        scoped tag
; comment          persisted rationale
```

The central mental model is simple: write one claim, append rather than overwrite, ask with the same shape using variables, and keep provenance and uncertainty explicit. Every larger subsystem in CAVE is built from that foundation.